Module 4: Database Schemas & Normalization

Comprehensive Study Guide: Architecture, Key Constraints, ER Modeling, and Normal Forms

Course: Database Engineering & Architecture **Document Ref:** Module 4 Note

1. What is a Database Schema?

A **Database Schema** is the formal blueprint that defines the logical structure, organization, and constraints of data within a relational database management system (RDBMS). It governs how tables are structured, how fields relate to one another, and how data integrity rules are enforced across the database environment.

Components of a Database Schema

- **Tables (Entities):** Structured containers consisting of rows and columns storing related record items.
- **Fields / Columns (Attributes):** Individual data points defined with explicit data types (e.g., `VARCHAR`, `INT`, `DATE`).
- **Keys & Constraints:** Rules governing data integrity, including `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, and `CHECK`.
- **Views & Stored Procedures:** Virtual tables built on saved SQL queries and executable business logic modules within the engine.
- **Relationships:** Explicit structural links binding child tables to parent primary key entities.

Key Advantages of Database Schemas

Advantage	Architectural Benefit
Management	Provides a structured hierarchy that simplifies database administration, maintenance, and refactoring without corrupting existing records.
Accessibility	Optimizes execution plans and simplifies standard SQL querying by establishing known object names and predictable paths.
Security	Enables fine-grained access control (RBAC). Administrators grant user permissions at the schema level rather than on raw tables.
Ownership	Allows grouping of database objects under logical owners, isolating applications sharing the same instance.

2. RDBMS Schema Implementation Examples

Database engines implement schemas with minor variations in syntax and structural container logic:

ENGINE-SPECIFIC SCHEMA IMPLEMENTATIONS

Microsoft SQL Server Schema Example

In SQL Server, schemas act as explicit namespaces inside a physical database container.

```
CREATE SCHEMA Sales AUTHORIZATION dbo;
GO

CREATE TABLE Sales.Orders (
    OrderID INT PRIMARY KEY IDENTITY(1,1),
    OrderDate DATETIME NOT NULL,
    TotalAmount DECIMAL(10,2) CHECK (TotalAmount >= 0)
);
```

PostgreSQL Schema Example

PostgreSQL databases automatically create a `public` schema by default, but allow custom schemas within the search path.

```
CREATE SCHEMA inventory;

CREATE TABLE inventory.products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) CHECK (price > 0)
);
```

Oracle Database Schema Example

In Oracle, a schema is automatically created for and bound to a specific database **User Account**.

```
-- In Oracle, creating a user implicitly creates their dedicated schema
CREATE USER app_admin IDENTIFIED BY SecurePassword123;
GRANT CREATE SESSION, CREATE TABLE TO app_admin;

-- Objects created under app_admin belong to the app_admin schema
CREATE TABLE app_admin.customers (
    customer_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    customer_name VARCHAR2(100) NOT NULL
);
```

3. Types of Database Schemas

Database design moves through three distinct visual and conceptual levels during the software development lifecycle:

Schema Type	Description	Target Audience / Focus
1. Conceptual / Logical Schema	Defines table structures, columns, and logical business rules independently of any technical storage engine hardware.	Database Architects & Business Analysts
2. Entity-Relationship (ER) Schema	Visual diagram depicting entity boxes, attributes, and explicit cardinalities connecting entities using formal notation.	Software Engineers & Data Engineers
3. Physical Schema	Translates the logical model into concrete implementation code: disk allocations, index structures, partitions, and field types.	Database Administrators (DBAs)

4. Practical Schema Example: 3 Connected Tables

Consider a standard e-commerce transaction model containing three relational tables: **Customers**, **Orders**, and **Invoices**.

RELATIONAL FLOW ARCHITECTURE

5. Table Relationships & Cardinality Shapes

Relationships define how records in one table correspond to records in another table.

Relationship Type	Cardinality Description	ER Diagram Shape Notation
One-to-Many (1:N)	A single record in Table A links to multiple records in Table B, but a record in Table B links to only one in Table A. *(Most Common)*	Solid line ending with a **Crow's Foot** at the "Many" side.
One-to-One (1:1)	A single record in Table A connects strictly to a single corresponding record in Table B. Often used for security or splitting large tables.	Single vertical tick mark or straight line at both ends.
Many-to-Many (M:N)	Multiple records in Table A link to multiple records in Table B. Requires a **Junction** / **Bridge Table** in RDBMS engines.	Crow's feet on both sides pointing to an intermediate junction entity box.

6. Key Concepts & Definitions

Primary, Candidate, and Composite Keys

- **Candidate Key:** Any attribute (or set of attributes) capable of uniquely identifying a record in a table without nulls.
- **Primary Key (PK):** The single Candidate Key selected by the DBA/architect to uniquely index and identify every table row.
- **Composite Key:** A Primary Key formed by combining two or more attributes together when no single attribute uniquely identifies the row (common in junction tables).

Foreign Keys (Parent vs. Child Tables)

A **Foreign Key (FK)** is an attribute in a **Child Table** that references the Primary Key of a **Parent Table**, enforcing referential integrity.

> **Referential Integrity Rule**
> You cannot insert a Foreign Key value into a child table unless that exact value already exists inside the primary key column of the parent table.

Entities & Attributes

- **Entity:** A real-world object, person, place, or event about which data is stored (represented as a Table row set).
  How to find it: Look for nouns in business requirements (e.g., `Customer`, `Invoice`, `Product`).
- **Types of Attributes:**
  - **Simple / Atomic:** Cannot be further subdivided (e.g., `Age`, `Price`).
  - **Composite:** Can be broken into smaller sub-components (e.g., `Full Address` = `Street` + `City` + `Zip`).
  - **Single-valued:** Holds only one value per record (e.g., `Social Security Number`).
  - **Multi-valued:** Can store multiple values for an entity (e.g., `PhoneNumbers`). Must be split in 1NF.
  - **Derived:** Calculated on the fly from other attributes (e.g., `Age` calculated from `DateOfBirth`).

7. Database Normalization & Anomaly Challenges

Normalization is the systematic process of organizing data in a database to reduce data redundancy, eliminate duplicate attributes, and enforce data integrity across tables.

The 3 Modification Anomalies (Un-normalized Data Risks)

1. Insertion Anomaly **Problem:** Inability to insert valid data into an entity without being forced to populate unrelated attributes (e.g., unable to add a new course until a student enrolls in it).

2. Update Anomaly **Problem:** Data inconsistency resulting from partial updates. If a customer address changes and exists in 10 different rows, updating 9 rows creates conflicting data.

3. Deletion Anomaly **Problem:** Accidental loss of critical data when deleting a record. (e.g., deleting a student enrollment record unexpectedly wipes out the course details entirely).

8. The Normal Forms (1NF, 2NF, 3NF)

Functional Dependency vs. Partial Dependency

- **Functional Dependency ($X ightarrow Y$):** An attribute Y is functionally dependent on X if the value of X uniquely determines the value of Y.
- **Partial Dependency:** Occurs when a non-key attribute depends on only *part* of a Composite Primary Key, rather than the entire key.

1st Normal Form (1NF)

Rules: Each column must contain only atomic (indivisible) values, and repeating groups or arrays must be eliminated.

Un-normalized Table (Non-atomic values in Products):

OrderID	Customer	Products (Non-Atomic)
101	Alice	Laptop, Mouse

1NF Compliant Table:

OrderID	Customer	Product
101	Alice	Laptop
101	Alice	Mouse

2nd Normal Form (2NF)

Rules: Must be in 1NF **AND** must eliminate all **Partial Dependencies**. Every non-key attribute must depend on the *entire* Primary Key.

> **2NF Resolution Strategy**
> If a table has a single-column Primary Key, it is automatically in 2NF! If it has a Composite Key, move partially dependent columns into a separate lookup table.

3rd Normal Form (3NF)

Rules: Must be in 2NF **AND** must eliminate all **Transitive Dependencies** ($X ightarrow Y$ and $Y ightarrow Z$). Non-key attributes must depend *only* on the Primary Key, nothing else.

SUMMARY OF NORMALIZATION PROGRESSION

1NF: Make column values atomic (No repeating groups or comma lists).

2NF: Remove partial dependencies (Non-keys depend on the WHOLE primary key).

3NF: Remove transitive dependencies (Non-keys depend ONLY on the primary key, directly).